Tattvik

What is Tattvik

Tattvik is an AI powered learning platform designed for students from Grades 6 to 10.

It follows actual school textbooks instead of random online content. It supports CBSE, ICSE, and IB boards. The platform is used by students, teachers, schools, parents, and platform administrators.

The Core Problems It Solves

Students do not revise consistently.

Students only realize what they do not know when exams are near.

Teachers cannot easily track every student’s progress.

Parents do not have visibility into what their child is learning.

Traditional one size fits all teaching does not work for everyone.

Who Uses Tattvik

Platform Admin

Handles the overall platform setup and governance.

MVP responsibilities:

- Set up boards such as CBSE and ICSE
- Configure grades, subjects, and textbooks
- Onboard schools

- Enable supported languages such as English and Tamil
- Copy academic calendars from previous years

Later phase:

- Add audio explanations in different languages
- Manage subscriptions and billing
- Track platform usage across schools
- Generate comparative reports across boards and schools

Platform Teacher (Subject Expert)

Responsible for content quality and academic integrity.

MVP responsibilities:

- Upload textbook PDFs and verify extracted content
- Add validated questions to the question bank
- Review and approve AI explanations
- Configure daily practice patterns
- Create and manage teaching schedules

Later phase:

- Improve AI responses based on feedback
- Review flagged AI answers
- Continuously refine content quality

School Admin

Manages the platform at the individual school level.

MVP responsibilities:

- Bulk upload teachers and students
- Override the board calendar for school specific events
- Revert to board calendar when needed

Later phase:

- View usage statistics
- Track academic performance across grades
- Export reports

School Teacher

Focuses on classroom level management.

MVP responsibilities:

- Map topics to exams
- Remove only topics they personally added
- View who added which topic
- Override class teaching schedules
- Revert to board schedules if required

Later phase:

- Override AI grading and add feedback

- View class progress snapshots
- Upload extra notes and resources
- Record and share class notes
- Receive alerts when students fall behind

Student

The primary user of the platform.

Learning

- Browse textbooks by lesson and topic
- Revise with AI step by step explanations
- Ask doubts while revising
- Choose preferred language
- Use conversational search

Practice

- Complete daily practice for each topic
- Attempt exam practice papers
- Generate extra practice papers if needed
- Receive instant AI grading
- See detailed explanations for mistakes

Tracking

- View completion percentage per topic

- Receive reminders for pending work
- Get AI suggestions based on upcoming exams
- Create notes linked to topics
- Use text to speech support

Guided Chat

Separate from topic revision.

- Helps with essays, research, and homework
- Modes include Explore, Research, and Schoolwork
- Does not provide full answers
- Breaks tasks into smaller steps
- Encourages independent thinking

Later:

- Receive automatic notifications when teachers mark topics as taught

---

Parent

Phase 2 feature.

- Link to child’s account
- View chat history in read only mode
- Track learning progress
- Enable strict mode if needed
- Manage linked students

- Receive weekly summary emails in the future

NGO Partner (Future)

Low priority feature.

- Access anonymized student data
- Download impact reports

Core Features

Textbooks and Lessons

Structured hierarchy:
Board → Grade → Subject → Lesson → Topic

Teaching Schedules

- Automatically generated calendars
- Adjustable by teachers
- Tracks holidays and exam dates
- Override system with revert option

AI Revision Help

- Step by step explanations

- Age appropriate language
- Interactive doubt resolution

Daily Practice

- Auto generated questions
- AI grading
- Weak area identification

Exam Practice

- Timed mock tests
- Topic wise question papers
- Readiness tracking

Progress Tracking

- Students see completion percentages
- Teachers see class level performance
- Parents see individual student progress

Guided AI Chat

- Supports research and writing

- Encourages thinking instead of copying
- Monitored by parents when enabled

Multi Language Support

Currently supports English and Tamil.

More languages planned.

Learning Flow

Teacher sets the schedule.

Student learns a topic.

AI supports revision.

Student practices at least twice.

Student takes exam practice.

Progress is visible to all stakeholders.

Cycle repeats.

Architecture Overview

Tattvik does not use a traditional Node.js backend for most operations.

Most data operations go directly from the frontend to Supabase.

Architecture flow:

UI → Hooks → Services → Repository → Supabase → PostgreSQL

This keeps the structure clean and layered.

AI Services Setup

Frontend AI Services

Handled directly from the browser.

- General chat using Gemini
- Learning recommendations using Gemini
- Conversational onboarding chat using Gemini or OpenAI

These expose API keys in browser requests, which creates security and scaling concerns.

Backend AI Services

Handled securely through a Python FastAPI backend.

- AI revision chat via WebSocket
- Concept generation server side
- API keys remain secure

This setup improves security, monitoring, and rate limiting.

Layout System

The UI is powered by a central layout system.

AppLayout.tsx detects device type and renders the correct layout.

RoleBasedLayouts.tsx passes user role to AppLayout.

Device detection flow:
useResponsiveLayout → useDeviceType → screen width check

Navigation components:

- RoleBottomNavigation for mobile
- HamburgerMenu for mobile side menu
- RoleSidebar for tablet and desktop
- RoleBranding for header and logo
- FloatingChatButton for student mobile chat access

Key Principles

MVP features are built first.

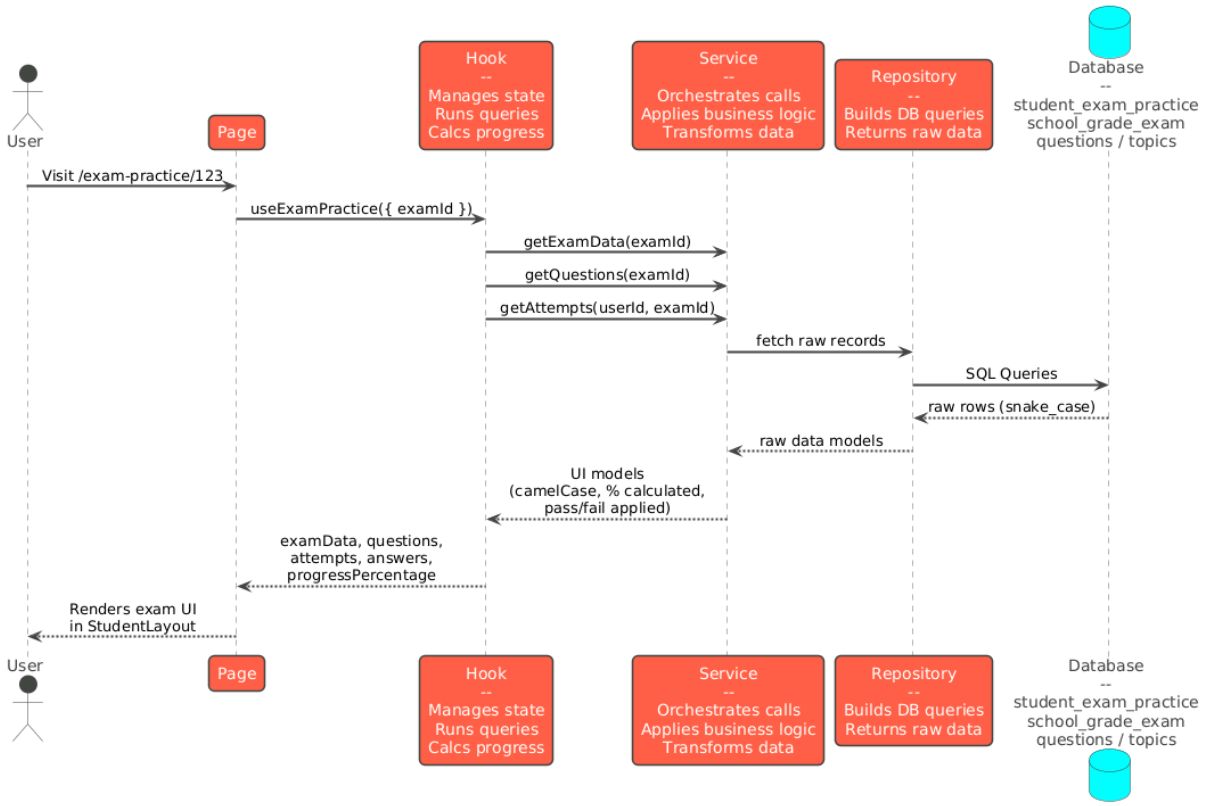
Everything else is phase 2 or later.

Ownership rules prevent conflicts between teachers.

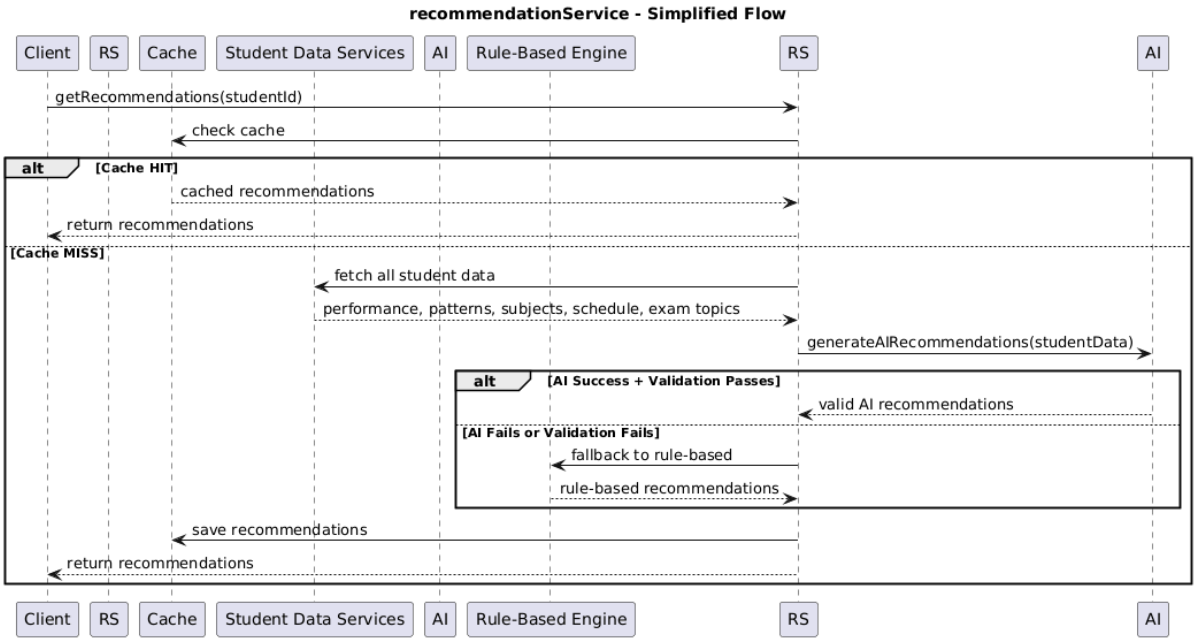
Override systems always allow reverting to original templates.

AI guides students instead of giving direct answers.

How a function call works



How recommendation service works



How I Approached the Codebase

The first thing I did was skim through the entire GitHub repository. I went folder by folder to understand how everything was organized. The structure was very clean and easy to follow.

The overall architecture follows this pattern:

UI → Hooks → Services → Repository → Database (Supabase)

So the flow is basically:

- UI components handle rendering
- Hooks manage state and data fetching
- Services contain business logic and data transformation
- Repository layer talks directly to Supabase
- Supabase handles the PostgreSQL database

After that, I asked Copilot a lot of questions about different parts of the repository. That helped me understand the technical decisions and the overall workflow of the app.

Backend Observation

One major thing I noticed is that the app does not have a traditional Node.js backend.

Most of the calls to Supabase are being made directly from the frontend. Since Supabase is a Backend as a Service, it handles authentication, database access, and some backend logic. So instead of having a custom Express or Node server in between, the frontend communicates directly with Supabase.

That simplifies development, but it also raises some concerns.

AI Services in the App

The AI services are split into two categories.

Frontend AI Services

These are called directly from the browser.

- General Chat
  Uses Google Gemini API
  Called directly from the browser
  API key is exposed

- Learning Recommendations
  Uses Google Gemini API
  Called directly from the browser
  API key is exposed

- Conversational UI
  Uses Gemini or OpenAI
  Used for onboarding chat
  API key is exposed

Since these calls are made from the browser, the API keys are visible in network requests. That creates security risks because anyone can inspect the requests and misuse the keys.

It also creates scaling concerns. If users start making a lot of requests, you have to handle rate limiting carefully. Without a server layer, you have less control over throttling, monitoring, and abuse prevention.